WEEK 03 · ARCHITECTURE DECISION RECORD

00:00–00:40 · 0:40

EVIDENCE VERIFIED · DELIVERY PENDING

From fast-but-ephemeral to durable-at-least-once

RabbitMQ, a separate Worker Service, SQL-backed progress, and an explicit dead-letter path.

Persist bulk jobs in SQL, dispatch a small versioned command through RabbitMQ, and process it in a separately deployable worker.

Commit, then ACK

Manual acknowledgement follows durable SQL progress.

3 attempts

Bounded application retry, then a named DLQ.

At least once

Duplicate risk is named; exactly-once is not claimed.

DECISION ASK

Accept ADR 004 as the Week 3 bulk-processing architecture.

SPEAKER NOTES
40 SECONDS

The decision is to move bulk execution out of the API. The API persists the job and items, RabbitMQ carries a small job-reference command, and a separate worker processes it. We acknowledge only after durable progress. Failed processing gets three attempts and then a named DLQ. The ask is acceptance with at-least-once semantics—not an exactly-once claim.

Transition: submission speed is already good; the change is about what happens when a process fails.

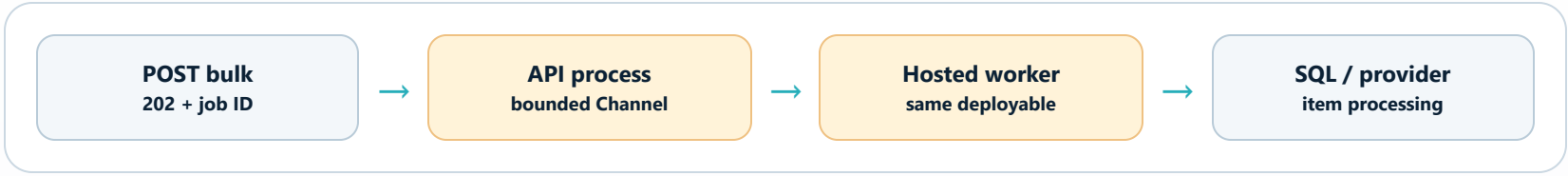
CONTEXT · WEEK 2 BASELINE

Fast at the edge. Fragile underneath.

The asynchronous HTTP contract works; the execution and status state still belong to one API process.

00:40–01:50 · 1:10

PREPARED · DELIVERY PENDING



GET status ← process-local job dictionary · lost with the API process

RECORDED SUBMISSION P95

2.004 ms

The queue decision is changing for recovery and isolation—not because the 202 path is slow.

Restart loss
Queued commands and status vanish with the API.

Replica split
Each API instance owns a different status dictionary.

No quarantine
Poison work has no named inspection path.

SPEAKER NOTES
70 SECONDS

Week 2 established the public contract: 202, job ID, status URL, and a measured p95 of 2.004 milliseconds. The compromise is that the channel and status dictionary live inside the API. Restart can lose both; a second replica sees different state; the hosted worker cannot deploy independently; poison work has no destination. ADR 003 documented these limits and deferred the broker boundary to Week 3. Its `IEmailSender` and provider-resilience decisions remain.

Transition: the drivers are recovery, isolation, and observability while preserving the API contract.

Evidence: evidence/week-02/job-lifecycle.md

ADR 003 queue portion is superseded

DECISION DRIVERS

Preserve the contract. Move the failure boundary.

Success is defined by recoverability and traceability, with the Week 4 scope boundary kept visible.

01:50–03:10 · 1:20

PREPARED · DELIVERY PENDING

1 — Keep the 202 contract
Fast submission, job ID, and status polling remain.

2 — Share durable status
Queryable across API and worker restarts.

3 — Separate deployment
Scale API and Worker Service independently.

4 — Bound failure
Redeliver unacknowledged work; quarantine poison.

5 — Correlate end to end
HTTP → broker → worker → SQL → DLQ.

6 — Minimize PII
Identifiers in RabbitMQ; item content remains in SQL.

7 — Run locally
Complete topology on Docker Desktop.

✓ — Stay honest
At-least-once with named gaps, never exactly-once.

SCOPE CONSTRAINT **Week 3 uses direct SQL + broker publication. Outbox, Inbox, and idempotency are Week 4.**

SPEAKER NOTES 80 SECONDS

The HTTP contract does not change because execution moves out of process. SQL becomes the shared status source. RabbitMQ must redeliver unacknowledged work, bound failure, and preserve stable job, message, and correlation identifiers. The command carries no recipient, subject, or body; the worker loads items from SQL. That reduces PII duplication at the deliberate cost of a shared database boundary. The scope constraint matters: direct SQL and broker writes are not atomic in Week 3.

Transition: evaluate each option against these specific drivers.

OPTIONS AND TRADEOFFS

RabbitMQ wins on the Week 3 drivers—not by default.

The selected option earns its added operational cost through isolation, acknowledgement, redelivery, and dead-lettering.

03:10–05:00 · 1:50

PREPARED · DELIVERY PENDING

OPTION	RESTART DURABILITY	INDEPENDENT WORKER	DLQ MODEL	COMPLEXITY	OUTCOME
In-memory `Channel<T>`	No	No	None	Low	Reject beyond W2
SQL polling queue	Yes	Yes	Custom	Medium	Reject for W3
RabbitMQ + SQL status	After broker acceptance	Yes	Native DLX / DLQ	Medium	SELECT
Kafka-style stream	Yes	Yes	Different model	High	Reject

Why it fits: competing consumers, manual ACK, redelivery, direct routing, DLX/DLQ, and Docker-local operation are native concepts.

What it costs: broker credentials, topology ownership, health/storage monitoring, eventual consistency, and a larger test surface.

SPEAKER NOTES
110 SECONDS

The channel is cheapest but fails restart, scaling, and DLQ needs. SQL polling can be durable, but it makes the team build leasing, polling, backoff, and dead-letter behavior on the business database. RabbitMQ fits a command work queue: competing consumers, manual acknowledgement, redelivery, routing, and dead-lettering are native, and it runs locally. Kafka provides durable event history and high-throughput streaming, but that is not this job-command problem. RabbitMQ is selected with its infrastructure and eventual-consistency costs stated, not hidden.

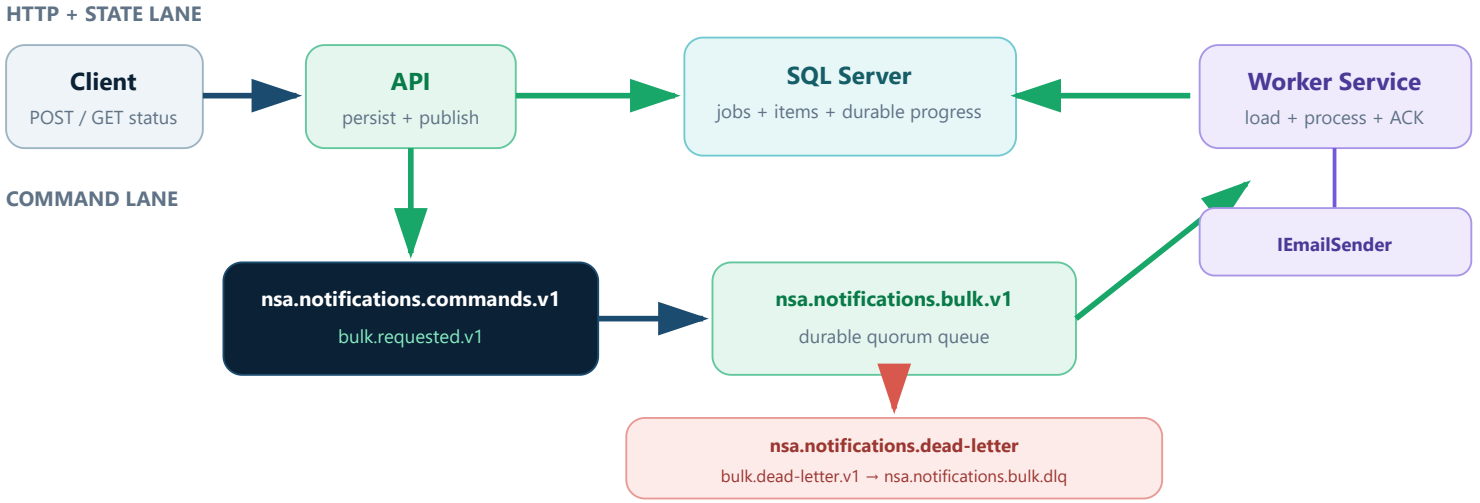
Transition: show the exact topology and where each piece of state lives.

A small command moves; business data stays in SQL.

Stable names make the path operable, testable, and easy to correlate.

05:00–07:10 · 2:10

PREPARED · DELIVERY PENDING



Command v1

Persistent JSON message · stable IDs across retry · no recipient, subject, or body

```
{
  "schemaVersion": 1,
  "messageId": "…",
  "jobId": "…",
  "correlationId": "…",
  "createdAtUtc": "…"
}
```

RabbitMQ.Client 7.2.1 · mandatory routing · publisher confirms · persistent

SPEAKER NOTES
130 SECONDS

The API persists the job and items, then publishes a persistent v1 command to `nsa.notifications.commands.v1` with `bulk.requested.v1`. The durable quorum queue is `nsa.notifications.bulk.v1`. SQL remains the status source while the worker loads items and records progress. The command contains only schema version, message ID, job ID, correlation ID, and creation time; IDs stay stable through retry. Final rejection uses `bulk.dead-letter.v1` through `nsa.notifications.dead-letter` into `nsa.notifications.bulk.dlq`. The client is RabbitMQ.Client 7.2.1.

Transition: topology is only half the choice—ACK ordering defines the guarantee.

DELIVERY AND FAILURE SEMANTICS

Commit first. ACK second. Name the duplicate risk.

A bounded retry path makes failures visible; it does not turn two systems into one transaction.

07:10–09:10 · 2:00

PREPARED · DELIVERY PENDING

SUCCESS PATH

Receive → Load job → Process → Commit SQL → Manual ACK

Crash before ACK: RabbitMQ redelivers. We prefer a possible duplicate over silent loss.

APPLICATION ATTEMPT BUDGET

1	`0 / absent`	retryable → republish as `1`
2	`1`	retryable → republish as `2`
3	`2`	reject, no requeue → DLQ

Malformed JSON or unsupported schema is permanent: reject without spending three attempts.

Guarantee: at-least-once

Publisher confirm ≠ atomic SQL + publish
Provider side effect before ACK may repeat

SPEAKER NOTES
120 SECONDS

Automatic ACK is disabled. The worker commits progress before acknowledging; if it dies first, RabbitMQ can redeliver. Missing `x-retry-count` means zero. Two retryable failures republish the same logical command with counts one and two. A third failure records safe terminal state and rejects without requeue. Malformed or unsupported commands dead-letter immediately. Confirms and mandatory routing provide transport evidence, but retry-publish/ACK and SQL/publish are still non-atomic. A provider success before durable progress can repeat. Broker attempts and Polly HTTP attempts are separate metrics.

Transition: connect the semantics to the captured restart, poison, and negative-path evidence.

RECORDED TECHNICAL EVIDENCE

Technical demo evidence: verified.

The target scenarios passed. Live presentation delivery and reviewer feedback remain pending.

09:10–11:30 · 2:20

DEMO VERIFIED · DELIVERY PENDING

✓ **76 / 76 tests passed · SQL Server, RabbitMQ, API, and worker healthy in hardened Compose**

1 Happy path · PASS

Submitted job reached `Completed`.

2 Worker redelivery · PASS

Paused worker: unack = 1. Same job `Completed` after force-kill/restart.

3 Broker recovery · PASS

Consumer recovered after restart; existing DLQ work was retained.

4 Bounded poison · PASS

`ae6d260a-8618-4477-84ef-97362b43fe1c`

3 attempts; retry = 2; x-death = rejected.

5 DLQ inspection · PASS

RabbitMQ screenshot: Ready = 5 and Total = 5.

6 Negative paths · PASS

503 `PublishFailed`, malformed JSON, unsupported schema/type, zero-match log review.

Evidence boundary: the original restart run proved recovery and DLQ retention; the final rerun separately proved survival of one unacknowledged main-queue command.

External status: technical demo complete; presentation delivery, reviewer decision, and feedback remain pending.

SPEAKER NOTES
140 SECONDS

The final rerun finished with 76 of 76 tests passing and all four hardened Compose services healthy. A happy job completed. Pausing the worker produced one unacknowledged delivery; the same job completed after force-kill/restart. RabbitMQ restart recovered the consumer and retained existing DLQ work, while a separate run preserved an in-flight command through broker restart. Poison job `ae6d260a-8618-4477-84ef-97362b43fe1c` ran three attempts, ending with retry count two and x-death rejected; the UI showed Ready 5 / Total 5. HTTP 503 PublishFailed, malformed JSON, unsupported schema, wrong AMQP type, and zero-match log review also passed.

Transition: these recorded paths passed; the design's declared consistency gaps still remain.

CONSEQUENCES AND BOUNDARY

We trade invisible loss for operable, explicit failure.

The architecture improves recovery and isolation while preserving a clear path to Week 4 correctness.

11:30–12:30 · 1:00

PREPARED · DELIVERY PENDING

✓ We gain

- Restart redelivery and SQL-backed status
- Independent API / Worker Service deployment
- Named poison quarantine and correlation
- Small broker commands without notification PII

△ We pay

- RabbitMQ topology, storage, health, and credentials
- Eventual consistency and more failure modes
- Duplicate-side-effect risk
- SQL/publish and retry-publish/ACK gaps

WEEK 3 Process isolation + bounded failure →

Week 4: Outbox reconciliation + Inbox/idempotency

SPEAKER NOTES 60 SECONDS

The design moves from invisible process-local loss to recoverable and inspectable work, and it adds broker and schema operations. Duplicate side effects and the SQL-to-broker dual write remain the central risks. Durable queues and transport confirms do not make two systems one transaction. Week 4 adds an Outbox that reconciles confirmed publication, followed by Inbox/idempotency for duplicate delivery. This is an incremental decision, not a perfection claim.

Transition: ask whether this is the right trade at the Week 3 boundary.

APPROVAL AND Q&A

Is this the right Week 3 trade?

Reserve 2 minutes 30 seconds for challenge, clarification, and a recorded outcome.

12:30–15:00 · 2:30

DECISION + FEEDBACK PENDING

Do we accept RabbitMQ, a separate Worker Service, SQL-backed status, manual ACK, and a three-attempt DLQ policy—while deferring atomic publication and deduplication to Week 4?

□ Accept ADR 004	□ Accept with named follow-up	□ Revise and re-review

Broker unavailable?
Verified: HTTP 503 + safe PublishFailed state; the dual-write gap remains until Outbox.

Can email duplicate?
Yes. A side effect before durable progress/ACK can repeat on redelivery.

Why job ID only?
Smaller messages and no recipient/subject/body copied into broker or DLQ.

Does DLQ recover work?
No. It quarantines work for an explicit inspect, repair, replay, or discard decision.

Why RabbitMQ over SQL?
Native ACK, redelivery, competing consumers, routing, and dead-lettering.

Does durable mean atomic?
No. Broker durability and confirms do not transact with the SQL commit.

SPEAKER NOTES 150 SECONDS

The claim is not that RabbitMQ makes delivery perfect. The claim is that restart recovery, independent execution, bounded failure, and observability justify the operational cost, with at-least-once semantics explicitly accepted. Invite challenge, then record one outcome: accept, accept with an owned follow-up, or revise and re-review. Do not fill in the feedback record until a live reviewer states the outcome. Current status remains prepared; delivery and feedback are pending.